

Unknot Recognition Workbench

Exact Reference Code, a Polynomial Hierarchy Subroutine,
and the Remaining Quasi-polynomial Obligations

Implementation and analysis prepared for Vladimir Reshetnikov

17 September 2026

Delivery status. The requested general $n^{O(\log n)}$ implementation has **not been completed**. The executable recognizer in this archive is exact on completion and has exponential worst-case complexity. A genuine polynomial-time terminal boundary-pattern routine and a conditional hierarchy bound auditor are also implemented. Neither supplies the missing compressed geometric hierarchy engine. No quasi-polynomial guarantee is claimed.

Abstract

This report documents working, standard-library Python code rather than a flowchart with assumed geometric oracles. The exact reference recognizer computes reduced Khovanov homology over $\mathbb{F}_2$, with an optional determinant obstruction. A separate routine decides essentiality of a boundary pattern on an already known 3-ball by enumerating edge bonds of size at most three and constructing dual-cycle witnesses. We prove its combinatorial criterion and polynomial complexity. We also give the precise conditional arithmetic behind $L(g+1)^L$, distinguish iteration counts from bit-operation bounds, and identify the unimplemented constructions needed to turn the supplied talk into the requested algorithm. The delivered test run passes 49 unittest methods. An explicit family of unknot diagrams demonstrates exponential state growth in the reference recognizer, so the implementation's limitation is not concealed by small-instance timings.

Contents

1	Scope, interface, and actual deliverables	3
2	What the supplied notes support	3
3	Validated diagram representations	4
4	The exact exponential reference recognizer	5
5	Running time: an explicit exponential lower bound	7
6	A polynomial terminal boundary-pattern algorithm	8
7	The hierarchy potential: a conditional executable audit	9
8	Concrete remaining implementation obligations	10
9	Testing, recorded results, and operational limits	11
10	Conclusion	12

1 Scope, interface, and actual deliverables

The input is a finite classical knot diagram with n crossings. The intended answer is whether its represented knot is ambient isotopic to the unknot. For conventional labels the diagram encoding has $O(n \log(n + 2))$ bits; arbitrary integer labels are also accepted, so parsing costs additionally depend on their bit lengths. The requested $n^{O(\log n)}$ bound concerns this *given* diagram, not the minimum crossing number of its knot.

The archive has three executable components:

Component	Implemented function	Established scope
recognize	Exact determinant filter and explicit reduced Khovanov chain complex	Exponential reference decider without ceilings
pattern	Essentiality on a known 3-ball; negative witness construction and checking	Polynomial in the encoded pattern size
bound	Integer potential, reset checks, conditional visit count	Arithmetic only; no topology inference

There is no recognizer backed by unspecified calls to “find a Cheeger region” or “build a hierarchical multi-surface.” There is also no hidden exponential fallback described as having the requested bound. The reference recognizer is explicitly a different algorithm. Its purpose is a runnable exact baseline and small-instance test oracle for subsequent geometric development.

Run from the archive root, without installing any third-party package:

```
python -m unknot_lab recognize examples/torus_3_5.json --check-d2
python -m unknot_lab pattern examples/pattern_prism.json
python -m unknot_lab verify-pattern examples/pattern_prism.json results/pattern_prism.json
python -m unittest discover -s tests -v
python benchmark.py
```

Python 3.10 or later is required by the package. Execution in this session used CPython 3.13.5 on Linux; other versions were not tested. JSON results from the recognizer explicitly contain `quasipolynomial_guarantee: false`.

2 What the supplied notes support

The supplied document is Marc Lackenby’s February 2021 talk, with 109 PDF pages, including progressive-reveal slides [1]. The main theorem is announced on pages 11–14. The basic hierarchy algorithm reaches its complete flowchart on page 53: construct surfaces, cut, find a violating disc, and either obtain a spanning disc or simplify an earlier surface and discard later ones.

The implementation-critical refinements are not merely different choices of loop syntax. Page 74 names four speed-ups: quadratic-complexity surfaces, compressed hierarchy encodings associated with Agol–Hass–Thurston, multi-surfaces, and Heegaard splittings with Cheeger regions. Pages 82–91 explain how multi-surfaces are intended to shorten the effective hierarchy; the surfaces represent independent relative homology classes and need not be pairwise disjoint. Pages 100–108

describe the Heegaard-based size measure and Cheeger-region alternative. The last page combines these ideas into a larger flowchart [1, pp. 74, 82–91, 100–109].

This terminology matters. The talk explicitly warns that its “genus” can mean an Euler-characteristic or pattern-sensitive complexity rather than ordinary genus [1, p. 59]. An implementation cannot substitute ordinary genus without proving that every relevant simplification decreases the chosen integer. Likewise, storing large multiplicities in binary does not itself implement cutting, component extraction, surface pullback, or normalisation on compressed objects.

Separate outside-source check. A July 2026 paper by Lackenby develops the hierarchy algorithm further. Section 9 separates the visit count $L(g+1)^L$ from unspecified costs of some steps. Proposition 10.2 subsequently bounds a refined hierarchy length by $4c_q(\mathcal{H})$; this is not the logarithmic-depth bound needed here. Its Proposition 2.6 supplies a dual-graph characterization of essential patterns on balls, which we use as an independent test criterion [2]. These are outside findings, not additional content attributed to the attachment.

Our practical attempt therefore implements the terminal graph problem exactly, while keeping the unimplemented geometric obligations visible in Section 8. This is not a claim that those obligations are impossible to discharge.

3 Validated diagram representations

3.1 Planar diagrams

A crossing is a tuple (a, b, c, d) of cyclically ordered edge labels. The underpassing strand joins a to c and the overpassing strand joins b to d . Each label occurs exactly twice. An empty crossing list denotes precisely one crossing-free circle. The standard trefoil fixture is [6]

```
{"pd": [[1,4,2,5], [3,6,4,1], [5,2,6,3]]}
```

The implementation rejects Boolean labels, noninteger labels, missing occurrences, and malformed crossing tuples. It first rennumbers labels densely; original labels are retained for basepoint selection and round trips.

For $n > 0$ there are $4n$ half-edges, also called darts. Let α exchange the two occurrences of each edge and let ρ rotate the four darts at each crossing. Face walks are the cycles of $\rho\alpha$. The underlying projection must be connected. Its orientable rotation surface has

$$V = n, \quad E = 2n, \quad \chi = V - E + F = F - n.$$

Consequently it is a sphere precisely when $F = n + 2$. This check rejects a valid abstract rotation system on a positive-genus surface rather than silently treating it as a classical knot diagram.

A second union-find joins opposite slots at every crossing. Its classes count link components. Exactly one class is required. Thus the detector theorem is never applied to an unsupported multi-component link. No unspecified crossing-free components are appended to a nonempty diagram.

3.2 Braid closures

The alternative input is a strand count and a signed Artin-generator word. The permutation induced by the word must be a single cycle. Before allocating an array indexed by strands, the parser uses

the necessary inequality $\# \text{letters} \geq \# \text{strands} - 1$; this rejects enormous, obviously invalid strand counts supplied in a short JSON input.

Each generator creates two new lower wire ends. Positive generators place the left upper strand over the right; negative generators reverse that choice. After the lower ends are joined to the corresponding upper ends to close the braid, the resulting PD passes through the same validation as an ordinary PD. This avoids having two recognition paths with different input assumptions.

4 The exact exponential reference recognizer

4.1 The chain groups and their explicit basis

Khovanov's construction and Bar-Natan's computational exposition give the underlying invariant [3, 4]. Our implementation discards absolute grading shifts and the quantum grading because only total reduced rank is needed. It does not replace homology by a graded Euler characteristic.

For each $s \in \{0, 1\}^n$, smooth crossing (a, b, c, d) by

$$s_j = 0 : \quad (a, b), (c, d), \quad s_j = 1 : \quad (a, d), (b, c).$$

Union-find computes the resulting circles as sets of original edge labels. Choose one marked edge, hence a distinguished circle in every resolution. Work over

$$A = \mathbb{F}_2[x]/(x^2).$$

The reduced basis labels the distinguished circle by x ; every other circle is independently labelled by 1 or x . A resolution with $k(s)$ circles therefore has $2^{k(s)-1}$ basis elements. The group in unshifted degree i is

$$C^i = \bigoplus_{|s|=i} \mathbb{F}_2^{2^{k(s)-1}}, \quad D_i = \dim_{\mathbb{F}_2} C^i.$$

Basis assignments are small integers whose bits select 1 or x on the unmarked circles. The marked circle is not a free bit.

4.2 Differentials

A directed cube edge changes a single zero to a one. Comparing component sets in its source and target determines which circles are unchanged and whether the local saddle merges or splits circles. The local maps are

$$\frac{(u, v)}{m(u, v)} \left| \begin{array}{cccc} (1, 1) & (1, x) & (x, 1) & (x, x) \\ 1 & x & x & 0 \end{array} \right. \quad \text{and} \quad \Delta(1) = 1 \otimes x + x \otimes 1, \quad \Delta(x) = x \otimes x.$$

Unchanged circles retain their labels. These formulas preserve the condition that the marked circle has label x . If a marked circle merges with another x -labelled circle, the image is zero, not an invalid reduced basis element.

Proposition 4.1. *The implemented maps satisfy $d^{i+1}d^i = 0$.*

Proof. A two-step path through the cube changes two distinct crossings. The two orders of changing them give the two sides of a square. For disjoint local saddles the maps act on different factors and

commute. In the interacting cases the needed identities are associativity of m , coassociativity of Δ , and

$$(m \otimes 1)(1 \otimes \Delta) = \Delta m = (1 \otimes m)(\Delta \otimes 1),$$

together with commutativity and cocommutativity. Each identity follows directly by evaluating its two sides on the $1, x$ basis. Thus both paths through every square induce the same map. Their sum is zero over $\mathbb{F}_2$. Every term in d^2 appears in exactly such a pair, and restriction to the reduced subcomplex preserves the identity. $\square$

No cube signs are needed over $\mathbb{F}_2$. For comparison with the integral invariant, the ordinary signed integral cube reduces to exactly these maps modulo two. The optional `-check-d2` forms differential columns and composes successive matrices. This is an additional execution-time consistency test, independent of the rank computation.

4.3 Bit-packed exact linear algebra

Each differential column is a Python integer. A set bit specifies a target basis vector. Terms cancel by XOR. Gaussian elimination selects the highest set bit as pivot; a dictionary stores one reduced column for each pivot. No floating-point arithmetic or numerical rank threshold is used.

Writing $r_i = \text{rank}(d^i)$ and $r_{-1} = r_n = 0$, the program computes

$$h_i = D_i - r_{i-1} - r_i, \quad H = \sum_{i=0}^n h_i.$$

Indeed, $\dim \ker d^i = D_i - r_i$, and the r_{i-1} -dimensional previous image is contained in that kernel. Negative dimensions or total rank zero trigger an internal-error result, never a knot verdict.

4.4 Why rank one is a complete decision criterion

The external topological input is Kronheimer–Mrowka’s rank-one theorem for reduced Khovanov homology [5]. The use of $\mathbb{F}_2$, rather than rational coefficients, can be justified without an additional detection conjecture.

Proposition 4.2. *For a classical knot K , $\dim_{\mathbb{F}_2} \widetilde{\text{Kh}}(K; \mathbb{F}_2) = 1$ if and only if K is the unknot.*

Proof. Take the signed reduced integral complex. Its groups are free abelian, so universal coefficients imply, degree by degree, that the homology dimension modulo two is at least the corresponding rational dimension. The total rational reduced homology is nonzero: otherwise its graded Euler characteristic, the reduced Jones invariant, would vanish, contrary to its normalization at 1. Thus total dimension one modulo two implies total rational dimension one. The rank-one theorem gives the unknot. Conversely, the crossing-free unknot has one reduced basis element and zero differential; invariance gives dimension one for every diagram of that knot. $\square$

The deep detector theorem is cited, not reproved or claimed to have been formalized. The code-level argument identifies the implemented complex with the usual construction, but the program has not been checked in a proof assistant.

4.5 A sound optional determinant filter

The knot determinant is the absolute Alexander evaluation at -1 [7]. The program constructs the Fox-colouring relation matrix: each row has 2 in the over-arc column and -1 in each under-arc column, with coefficients combined when columns coincide. A cofactor is evaluated using fraction-free Bareiss elimination and exact integer division.

A determinant different from one proves nontriviality. Determinant one does *not* prove triviality: it causes the program to continue to homology. The delivered $T(3, 5)$ example exercises exactly this branch. Its observed values are determinant one and reduced $\mathbb{F}_2$ rank seven.

5 Running time: an explicit exponential lower bound

Let $D = \sum_i D_i$. A smoothing of a connected n -crossing projection has at most $n + 1$ circles. To see this, join its circles at the original crossing locations: the resulting graph is connected and has at most n edges, so it has at most $n + 1$ vertices. Therefore

$$D = \sum_s 2^{k(s)-1} \leq 2^n 2^n = 4^n.$$

All saddle construction is polynomial in n per state or cube edge. Elimination on bit vectors of length at most D has a coarse $O(D^3)$ bit-operation bound, up to polynomial factors in n . Storing pivots takes at most $O(D^2)$ bits, again up to polynomial bookkeeping factors. Hence the unbounded reference implementation has a coarse $2^{O(n)}$ time and space bound, plus polynomial input-reading costs. Treating a Python integer as a constant-cost object would be incorrect; its vector bit length is included here.

Proposition 5.1. *The unbounded reference implementation does not have a $n^{O(\log n)}$ worst-case running-time bound, even with the determinant filter enabled.*

Proof. Consider the closure of

$$\beta_n = \sigma_1 \sigma_2 \cdots \sigma_n \in B_{n+1}.$$

Each last strand admits a Markov destabilization, so the closure is an unknot with n displayed crossings. Its determinant is one, and the filter cannot finish the computation. The cube constructor explicitly visits all 2^n states. This is already an $\Omega(2^n)$ operation lower bound, which exceeds $n^{C \log n}$ for every fixed C at sufficiently large n .

More precisely, in this family's input convention a state of weight i has $i + 1$ circles. This follows by removing the last single-crossing strand: one smoothing leaves the smaller diagram, the other also splits off a circle. Thus

$$D_i = \binom{n}{i} 2^i, \quad D = (1 + 2)^n = 3^n.$$

The growth is a feature of the specified implementation, not a conjecture about all possible unknot algorithms. $\square$

The benchmark checks these exact state and generator counts for $1 \leq n \leq 8$. The proof, not those finitely many timings, establishes the asymptotic limitation. With default ceilings the program instead eventually returns `unknown`; a time- or size-limited partial procedure is not a complete quasi-polynomial decider.

6 A polynomial terminal boundary-pattern algorithm

6.1 Domain and encoding

The notes define a boundary pattern P as embedded trivalent graphs and simple closed curves. On a ball it is essential when every disc boundary meeting P transversely at most three times bounds a boundary disc whose pattern is empty, a single arc, or a tripod [1, pp. 24–25].

The ambient manifold is assumed already to be a 3-ball. A vanishing first Betti number or a spherical boundary alone is not the required input certificate for an arbitrary manifold. The present code does not infer this assumption from such data.

Each graph vertex records a cyclic order of three half-edge IDs. IDs $2e$ and $2e + 1$ form edge e . Every ID must occur exactly once. Loops and parallel edges are permitted. The rotation surface of each component is checked to be a sphere. Components with no vertices are recorded by an integer circle count. For this particular decision problem, nesting of distinct components is irrelevant; the code does not claim to reconstruct that nesting.

6.2 The edge-cut criterion

For a connected cubic plane multigraph G , write $\delta(W)$ for the edges with one endpoint in a nonempty proper vertex subset W . A *bond* is a cut for which both shores induce connected subgraphs. A three-edge bond is *trivial* when one shore is a single vertex.

Theorem 6.1. *A boundary pattern on a 3-ball is essential precisely in the following cases: it is empty; it is one vertex-free circle; or it is a connected cubic plane multigraph with no one-edge bond, no two-edge bond, and no nontrivial three-edge bond.*

Proof. If the pattern has at least two connected components, a simple curve in their complement can separate some components from the others. Neither complementary disc has empty pattern, and the curve has no intersections with the pattern. Pushing the curve’s spanning disc into the ball gives a violating disc. Empty pattern and one circle satisfy the definition directly.

Suppose now that the pattern is a connected cubic graph. A bond in a plane graph corresponds to a simple cycle in its geometric dual: boundary arcs in the dual faces trace a separating curve that crosses precisely the bond edges. For a bond of size one no allowed disc pattern has that boundary-intersection count. For size two both shores contain vertices, so neither side is a single arc or empty. For a nontrivial bond of size three, neither side consists of one vertex and three exiting edges, so neither side is a tripod. These bonds therefore produce violations.

Conversely, suppose all three forbidden bond types are absent. Let C be a simple curve transverse to the pattern with at most three intersections. If one complementary disc contains no graph vertex, it contains at most one arc, or is empty. There is no detached circle in a connected cubic graph. Such a curve is not violating.

It remains that both discs contain vertices. Let W be the vertices on one side. Every edge of $\delta(W)$ crosses C , so $|\delta(W)| \leq 3$. Every nonempty edge cut contains a bond. Absence of one- and two-edge bonds therefore implies that every nonempty cut has size at least three. Consequently $|\delta(W)| = 3$, and each cut edge crosses C exactly once.

Both shores must be connected: if one had at least two components, each component would have an edge cut of size at least three, forcing at least six edges across that shore’s boundary. Thus $\delta(W)$ is

a three-edge bond. It must be trivial. Its singleton shore is a cubic vertex with three exiting edges and no internal loop, giving exactly an allowed tripod. Hence no violating curve exists. $\square$

The dual bond–cycle correspondence used here can also be seen by taking a small regular neighbourhood of a connected shore. Because the complementary shore is connected, the boundary component separating the two shores traces one dual cycle. A bridge yields a dual loop; a two-edge bond gives a dual two-cycle. These cases must not be removed by treating the dual as an ordinary simple graph before constructing witnesses.

6.3 Implementation and cost

The production algorithm examines all sets of one, two, then three edges. For each set it deletes those edges and computes connected components. Exactly two components with that actual cut define a bond. Size-three cuts with a singleton smaller shore are discarded as allowable tripods.

There are $O(E^3)$ candidate sets, and a connectivity scan costs $O(V + E)$. The decision procedure therefore costs

$$O(E^3(V + E))$$

combinatorial operations for a nonempty cubic graph. The encoding and integer indices introduce at most polynomial bit overhead. Empty and circle-only patterns are handled directly without expanding the circle count.

A negative graph certificate records a shore, its cut-edge IDs, and the ordered edges and faces of the dual cycle. The verifier recomputes the cut and checks connectivity and the cycle, without repeating the edge-subset search. For a disconnected pattern the witness is the verified component count: it certifies existence of a zero-intersection separating curve, not explicit geometric coordinates for one. An essential answer follows from exhaustive absence of small forbidden cuts, not from a claimed short positive certificate.

The tests independently form the dual and use the simple four-connected criterion, including the complete graphs on three and four vertices, from [2, Proposition 2.6]. This dual-vertex search differs from the production primal-edge search. All tested spherical rotations give the same answer.

7 The hierarchy potential: a conditional executable audit

The notes’ informal estimate treats surface complexities as bounded digits [1, pp. 60–65]. Here we state a precise arithmetic variant. Assume that complete phase states carry L nonnegative digits $d_1, \dots, d_L$, each at most g , with shorter lists padded by zero. Assume that a reset preserves all digits before some pivot j and strictly lowers d_j , while allowing arbitrary later digits within the same bound.

Put $B = g + 1$ and define

$$\Phi(d_1, \dots, d_L) = \sum_{i=1}^L d_i B^{L-i}.$$

This explicit radix handles the endpoint digit g ; the talk uses informal base- g notation. We do not silently interpret its unspecified “genus” as the concrete integer used by this module.

Lemma 7.1. *Every reset lowers Φ by at least one. The number of complete phases is at most $(g + 1)^L$. If each phase makes at most L loop visits, the visit count is at most $L(g + 1)^L$.*

Proof. For $g \geq 1$, a one-unit reduction of the pivot lowers the leading contribution by B^{L-j} . The largest possible increase from all later digits is

$$g \sum_{k=0}^{L-j-1} B^k = B^{L-j} - 1.$$

The net reduction is therefore at least one. Also $0 \leq \Phi \leq B^L - 1$, so at most B^L phase states can be visited in strict descending order. Multiply by the visits per phase. If $g = 0$, all digits are zero, no reset is possible, and the single-phase bound remains valid. $\square$

A hierarchy-building step that appends a surface can increase Φ . The comparison must be between complete phases surrounding a simplification and rebuild, not every individual instruction. The code's `verify_reset` checks the shared prefix, strict pivot decrease, digit bounds, and actual potential decrease. These are integer assertions, not computed topological invariants.

Theorem 7.2 (One sufficient quasi-polynomial contract). *Suppose an implemented geometric engine has uniform bounds*

$$g(n) \leq C(n+2)^a, \quad L(n) \leq b \log_2(n+2) + c,$$

its full-phase resets satisfy the preceding hypotheses, it has at most $R(n)$ outer restarts, and every visit costs at most $P(n)$ bit operations. If R and P are polynomially bounded, its total running time is $(n+2)^{O(\log(n+2))}$.

Proof. The total cost is bounded, up to constant control-flow overhead, by

$$P(n)R(n)L(n)(g(n)+1)^{L(n)}.$$

Its logarithm is $O(\log n) + O(\log n)O(\log n) = O((\log n)^2)$. Exponentiating gives the claim. All bounds must hold uniformly for actual stored states, including states after restarts. $\square$

These are sufficient hypotheses, not claims established for the delivered reference code. The theorem would also admit suitable quasi-polynomial per-visit or restart bounds, but merely asserting them does not implement anything. Likewise, a polynomial number of bits in a surface's normal vector does not establish a polynomial bound on operations that expand its discs.

The function `cheeger_inequality_only` evaluates

$$3g(\partial M_i) \leq \min\{g(H_i), g(H) - g(H_i)\}.$$

The notes additionally require the restricted Morse function to be a Heegaard Morse function [1, p. 108]. The numeric inequality alone neither recognizes such a region nor constructs the resulting modification.

8 Concrete remaining implementation obligations

The following are missing geometric constructions, not optional performance polish on existing code. Naming their outputs is not a substitute for constructing them.

Initial layered handle structure. Convert the diagram to the exterior with an explicit embedding model and layered handle/Morse data. Retain boundary identifications and peripheral information needed to recognize a compression disc for the original boundary. Establish the initial size and bit-length bounds. The current PD parser does not create this structure.

Compressed cuts and surface transport. Specify local cell types, attaching maps, multiplicity encodings, and interval pairings. Implement cutting and re-gluing, Euler-characteristic and component calculations, orientation, and maps back to earlier stages without expanding exponentially many parallel discs. Any appeal to compressed orbit counting must include the actual encoded operations and their cost bounds.

Hierarchical multi-surfaces. Construct the relative homology classes and their embedded representatives, including intersection data and the sequential cuts that convert them into hierarchy stages. Prove independence and the required complexity control. Compression must update the multi-surface representation and identify precisely which later data remain valid. These tasks correspond to named boxes on the last slide, but there is no implementation of them in this archive.

Geometric Cheeger regions and weak reduction. Determine the relevant level surfaces and their intersections with each submanifold. Verify the Morse/Heegaard hypotheses, not just an integer inequality. Construct the new generalized splitting, implement weak reduction and the use of Haken’s lemma, and bound all resulting restarts and changes of representation.

Terminal topology and witness pullback. Establish the hypotheses permitting the terminal regions to be treated as balls. Apply the implemented boundary-pattern subroutine there. Transport its disc witness through the hierarchy to either a genuine original-boundary compression or a justified earlier simplification. A dual-cycle witness on an abstract ball pattern by itself is not a knot spanning-disc certificate.

End-to-end complexity proof. Prove the appropriate global digit and depth bounds for the implemented objects, including the pattern-sensitive meaning of the digits. Account for normalization, large-integer arithmetic, compressed surface updates, and outer restarts. Only after these obligations are met would the conditional bound in Section 7 become a runtime theorem for an actual quasi-polynomial recognizer.

9 Testing, recorded results, and operational limits

The delivered `results/unittest.txt` records 49 passing test methods. Methods contain additional parameterized cases. The linear-algebra tests compare all 512 binary 3×3 matrices with a separate dense elimination routine, use 100 seeded random binary matrices, and compare 140 integer determinants of orders zero through six with rational Gaussian elimination.

Knot tests cover mirrors, every basepoint of the figure-eight example, relabeling, crossing reordering, half-turns of crossing tuples, inverse braid pairs, the braid relation, positive and negative stabilizations, and 24 seeded conjugates of unknot braid words. Homology computations in these tests check $d^2 = 0$. Pattern tests include all spherical choices of local rotations of several fixed multigraphs,

independent dual-criterion comparisons, and rejection of modified certificates. CLI tests check invalid inputs, resource limits, exact verdicts, and the standalone witness verifier.

Example	Crossings	Determinant	Reduced $\mathbb{F}_2$ rank	Verdict
Crossing-free circle	0	1	1	Unknot
One curl	1	1	1	Unknot
Trefoil	3	3	3	Nontrivial
Figure-eight	4	5	5	Nontrivial
$T(3, 5)$	10	1	7	Nontrivial
Conjugated unknot fixture	8	1	1	Unknot

These are recorded computations, not new knot-theoretic claims. Several ranks in the unit suite are fixed regression expectations. No independent complete Khovanov implementation was executed, and these tests do not formally verify the recognizer. The different dense rank and dual-pattern algorithms provide independent checks of their respective components, not of the entire program.

n in Proposition 5.1	Resolutions	Chain generators	Logical matrix bits
1	2	3	2
2	4	9	20
3	8	27	174
4	16	81	1,480
5	32	243	12,570
6	64	729	107,100
7	128	2,187	916,230
8	256	6,561	7,869,200

Timing and environment data are in `results/validation.json`; `benchmark.py` regenerates them. Timings are not used as complexity evidence.

Resource and error semantics. The default limits are 65,536 states, 200,000 generators, and 128,000,000 logical matrix bits, where the last quantity is $\sum_i D_i D_{i+1}$. It is a conservative combinatorial estimate, not a Python memory-usage bound. The program does not enforce a wall-clock or RSS limit. The `-unlimited` option removes all three ceilings and can be expensive. A stopped computation returns `unknown` with a null Boolean, never `nontrivial`. Exact yes and no answers both exit with code zero; resource stops exit two, invalid data three, and detected internal arithmetic errors four. Argparse also uses exit two for command syntax errors, with a message on standard error rather than a JSON mathematical result.

10 Conclusion

There is working exact recognition code, a proved polynomial terminal boundary-pattern algorithm, a conditional complexity auditor, examples, and a reproducible test suite in this archive. The unbounded recognizer is a complete exponential decision procedure, not a realization of the talk’s announced quasi-polynomial algorithm. Proposition 5.1 explicitly rules out that claim for this implementation. The missing geometric constructions and uniform bit-cost bounds are listed above so that subsequent work can address the actual requirements rather than inherit an unjustified runtime label.

References

- [1] Marc Lackenby. *Unknot recognition in quasi-polynomial time*. February 2021 talk slides. User-supplied file `quasipolynomial-talk.pdf`, 109 PDF pages. Page numbers in this report refer to those PDF pages, including reveal slides. Not redistributed in the archive.
- [2] Marc Lackenby. *Incompressible surfaces, hierarchies and unknot recognition*. arXiv:2607.23350v1, 25 July 2026. <https://arxiv.org/html/2607.23350v1>.
- [3] Mikhail Khovanov. *A categorification of the Jones polynomial*. arXiv:math/9908171. <https://arxiv.org/abs/math/9908171>.
- [4] Dror Bar-Natan. *On Khovanov’s categorification of the Jones polynomial*. Publication and computational exposition page. <https://www.math.toronto.edu/~drorbn/papers/Categorification/>.
- [5] Peter B. Kronheimer and Tomasz S. Mrowka. *Khovanov homology is an unknot-detector*. arXiv:1005.4346. <https://arxiv.org/abs/1005.4346>.
- [6] Knot Atlas. *The trefoil 3_1* . Standard planar-diagram fixture. https://katlas.org/wiki/3_1.
- [7] Knot Atlas. *The Alexander–Conway Polynomial*. Determinant convention. https://katlas.org/wiki/The_Alexander-Conway_Polynomial.